

Phase 1 (CRM & Project Management) — Implementation Plan

Date: 2026-07-08 **Depends on:** Phase 0 (Foundation) — merged to main at 48e1b4a. **Scope source:** docs/02-functional-requirements.md Sections 2–3, docs/09-roadmap-implementation-plan.md Phase 1, docs/04-database-schema.md Sections 3–4, docs/05-api-specification.md Sections 3–4.

Phase 1 Scope (from the Roadmap, verbatim)

- CRM: Lead CRUD, pipeline status transitions, communication logs.
- Project Management: Project CRUD/lifecycle state machine, Phases, Tasks, Documents, Daily Logs.
- LEAD_WON → draft Project event wiring.
- Client-facing read-only project dashboard (sanitized view, RBAC-scoped).
- **Exit criteria:** a Lead can be created, moved to Won, and land as a Draft Project with client details carried over, end-to-end, with tests.

Explicitly Out of Scope for Phase 1 (deferred, not forgotten)

- **Change Orders.** docs/02-functional-requirements.md US-3.6 tags Change Orders under the “Project Management (P1)” module header, but docs/09-roadmap-implementation-plan.md explicitly places “Change Order creation + e-signature approval” under **Phase 2**, alongside the Estimation Engine, because Change Orders “reuse the e-signature capability built for Estimates.” The change_orders table itself (docs/04-database-schema.md Section 4) has a hard FK to esignatures(id), which doesn’t exist until Phase 2. Building change_orders now would mean either a dangling/nullable FK to a table that doesn’t exist yet, or a change order flow with no way to make it binding — neither is useful. **Decision: change_orders is not built in Phase 1.** The roadmap’s phase bullets are the authoritative sequencing signal here, not the functional-requirements doc’s module-level priority tag.
- Estimation Engine, e-signatures, billing/invoicing (Phase 2+).
- Compliance/subcontractor tracking (Phase 3).
- Full async task queue (Celery/Dramatiq) — see design decision #2 below for what Phase 1 does instead.
- File storage beyond local disk (e.g. S3/object storage) — see design decision #4.
- Refresh-token rotation, MFA (carried over from Phase 0’s deferred list, still not triggered).

Inherited Invariants from Phase 0 (apply to every new table/route — do not rediscover these bugs)

These are load-bearing lessons from Phase 0's 10 design decisions (docs/superpowers/plans/2026-07-07-phase-0-foundation.md). Every task below assumes them; call them out explicitly in review if a task's diff doesn't follow one.

1. **Every RLS policy cast must be `NULLIF(current_setting('app.x', true), '')::uuid`, never a bare cast.** A bare cast raises an unhandled error once a pooled connection has ever seen that GUC set (design decision #7).
2. **Every UPDATE policy needs an explicit `WITH CHECK`, not just `USING`.** `USING` alone only gates which row you may target, not what you may write into it (design decision #6).
3. **New pre-tenant-context lookups need their own bootstrap RLS policy**, not a bare `tenant_isolation` policy that requires context that doesn't exist yet (design decisions #3, #9). Phase 1 has no obvious case like this (unlike invitation-acceptance), but if one turns up, this is the established pattern.
4. **Route handlers reuse `current.session` from `get_current_user/require_role`; never open a new session, never call `session.commit()` inline.** The dependency owns the transaction and commits once, after the handler returns (design decision #8).
5. **Migrations run as the postgres owner role; the app connects as `app_user`, which is subject to RLS.** Never grant `app_user` table ownership (design decision #1).
6. **Regression Testing Policy (Phase 0, carried forward unchanged):** from this point on, every task's verification runs the FULL test suite (`pytest -v`, no path filter), not just the new task's own test file.
7. **Every task goes through implementer → independent spec-compliance review → independent code-quality review**, each re-verifying claims against the live database, not trusting the prior report. This is not optional process theater — every single Phase 0 task from #5 onward found at least one real bug this way.

New Critical Design Decisions for Phase 1

1. **Module organization stays flat** (`backend/app/{routers,models,schemas,services}/`), not the `src/crm/`, `src/projects/` package split shown in [Technical Architecture Section 3](#). That doc's own encapsulation rule is explicitly enforced "by code review discipline... rather than by physical service boundaries" for now. Restructuring Phase 0's already-tested users-domain code into packages for no functional

gain, mid-project, is unnecessary churn. New files follow domain-grouped naming within the existing flat layout instead: `models/lead.py`, `models/project.py`, `routers/leads.py`, `routers/projects.py`, etc. Revisit the package split only if a real cross-module coupling problem shows up.

2. **The LEAD_WON event bus is in-process and synchronous for Phase 1, not Redis-backed.** [Technical Architecture](#) Section 4 says “in-process (or Redis-backed)” — Redis-backed pub/sub, and the Celery/Dramatiq worker it would imply, is explicitly tied in that same doc to *Phase 2+* concerns (PDF generation, QuickBooks sync, outbound email). Phase 1 has exactly one event (LEAD_WON → draft a Project) with one consumer, in the same request, same transaction. A synchronous in-process dispatcher (`app/core/events.py`: a `publish(event_name, payload)` that calls registered handlers directly, in-transaction) satisfies the architecture’s intent — decoupled publish/subscribe call sites — without standing up Redis/Celery for a single synchronous handler. The dispatcher interface is deliberately shaped so swapping the in-process call for a Redis-backed queue later doesn’t require touching call sites (`publish()`’s signature doesn’t change; only its internals would).
3. **PATCH `/projects/{id}` (general field edit: name, site_address, projected_start_date) is added beyond [API Specification Section 4’s literal route table](#)**, which only lists `PATCH /projects/{id}/status`. This is a real gap, not an oversight to preserve: a LEAD_WON-drafted Project (design decision #5 below) is created with an empty `site_address`, and the documented API gives no way to ever fill it in. The API spec doc’s own preamble says it “describes API contracts conceptually,” not exhaustively — this is a reasonable, necessary extension, not a deviation from intent.
4. **Document storage is local filesystem for Phase 1, under a Docker-volume-backed path, not S3/object storage.** No object storage service is defined anywhere in the architecture doc for the self-hosted deployment target; introducing one now for a single `POST /projects/{id}/documents` endpoint would be scope creep against Phase 1’s actual exit criteria. `documents.storage_path` stores a path relative to a configured root (e.g. `STORAGE_ROOT=/data/documents`, itself a new Docker volume in `docker-compose.yml`), and the upload endpoint writes the file under `{STORAGE_ROOT}/{company_id}/{project_id}/{filename}`. Revisit if/when a real deployment needs multi-host file access.
5. **LEAD_WON auto-drafts a Project with `site_address = ''` (empty string, not NULL) when the Lead carries no address.** `leads` (per [Database Schema](#) Section 3) has no address field, but

projects.site_address is NOT NULL. Rather than add an address field to leads (schema-doc scope creep) or make site_address nullable (weakens a documented NOT NULL constraint for every project, not just auto-drafted ones), the auto-draft path uses ' ' and leaves it to the PM to fill in via the new PATCH /projects/{id} (design decision #3) before advancing out of draft. This is called out explicitly so nobody “fixes” it later as an apparent bug without checking here first.

6. **Immutability for communication_logs and daily_logs is enforced at the database grant level, not just by omitting UPDATE/DELETE routes.** Both tables’ schema-doc comments already say “immutable by convention” / “immutable once submitted (application-layer enforced)” — Phase 0’s own culture (RLS as the *authoritative* mechanism, application code as a convenience, never the sole safeguard) argues for hardening this the same way: REVOKE UPDATE, DELETE ON communication_logs, daily_logs FROM app_user in the migration, so a future route added by mistake still can’t mutate history. documents doesn’t need this — new versions are new rows (version incremented), never an UPDATE of an existing row, so the same table-level revoke applies there too for consistency (REVOKE UPDATE, DELETE ON documents FROM app_user).
7. **RBAC role checks reuse require_role(*roles) from Phase 0 as-is** (app/core/deps.py) — it already accepts an arbitrary role list (e.g. require_role("admin", "project_manager")). No new dependency machinery is needed; only new call sites, matching the RBAC matrix in [Security & Compliance](#) Section 2.
8. **The client-facing “sanitized dashboard” is the same GET /projects/{id} route, not a separate endpoint,** per [API Specification](#) Section 4 (which lists only one GET /projects/{id}, “Full project dashboard data”). The response schema is selected by the caller’s role inside the handler: client role gets ProjectClientDashboardResponse (name, status, site_address, phase/task progress *without* assignee identities or internal notes — Phase 1 has no financial fields on projects yet, those arrive with Estimation in Phase 2, so “sanitized” here means “no internal operational detail,” not “no budget data”); every other role gets the full ProjectResponse. **Correction, found during Task 1.11’s spec review:** the original draft of this decision said “phase/task progress” but never specified concrete fields, and no task through Task 1.20 ever supplied that data — there’s no phase/task list route for a client to query independently (per the RBAC matrix, client only gets this one sanitized dashboard route), so without an explicit fix the dashboard would silently fail to satisfy this decision’s own stated requirement and [Functional Requirements](#) US-3.5 (“progress, milestones”). Fixed by adding three computed integer fields directly

to ProjectClientDashboardResponse: phase_count, task_count, completed_task_count (tasks.status == 'done'). Task 1.12's GET /projects/{id} handler computes these via COUNT queries scoped to the project (phases/tasks both already exist as of Task 1.10's migration, so this requires no new table and no dependency on Task 1.14's routes existing yet — the handler queries the tables directly, the same way every other router in this codebase queries tables without going through another router's endpoints).

Regression Testing Policy (unchanged from Phase 0, restated for this plan)

Every task's verification step runs `cd backend && pytest -v` (no path filter) after its own new/changed test file passes in isolation. A task is not done until the full suite is green. Any new migration additionally requires a full `test_tenant_isolation.py`-style adversarial pass for its new tables before being considered complete (see Task 1.12 and Task 1.20 below).

File Structure (additions to Phase 0's layout)

```
backend/
  migrations/versions/
    0003_crm_schema.py
    0004_project_management_schema.py
  app/
    core/
      events.py # in-process publish/subscribe
dispatcher (design decision #2)
  models/
    lead.py
    communication_log.py
    project.py
    phase.py
    task.py
    document.py
    daily_log.py
  schemas/
    lead.py
    communication_log.py
    project.py
    phase.py
    task.py
    document.py
    daily_log.py
  routers/
    leads.py
    projects.py
  services/
    lead_transitions.py # Lead status state machine +
LEAD_WON publish
```

```

    project_transitions.py      # Project status state machine
tests/
  test_leads.py
  test_lead_state_machine.py
  test_communication_logs.py
  test_projects.py
  test_project_state_machine.py
  test_phases_tasks.py
  test_documents.py
  test_daily_logs.py
  test_lead_won_drafts_project.py
  test_tenant_isolation_phase1.py  # cross-tenant regression for
every new table
scripts/
  e2e_smoke_test.py             # extended with the Lead→Won→Project
flow
docker-compose.yml             # + a documents volume (design
decision #4)

```

Task 1.1: Lead & CommunicationLog Models

Files: Create backend/app/models/lead.py, backend/app/models/communication_log.py. Modify backend/app/models/__init__.py.

- Lead: id (UUID PK), company_id (FK, not null), contact_name, project_name, email, phone (nullable), status (string, default "new"), estimated_value (Numeric(12,2), nullable), project_type, notes (nullable), created_at, updated_at. Match column types/nullability exactly to [Database Schema](#) Section 3.
- CommunicationLog: id, lead_id (FK, cascade), company_id (FK), author_id (FK to users), channel (string, CHECK-constrained at the DB level in the migration, not just Pydantic), body, created_at. No updated_at — immutable by design (design decision #6).
- Follow the exact SQLAlchemy model conventions established in app/models/company.py/user.py (mixins, type annotations, etc.) from Phase 0.
- Verify: pytest -v still green (no behavior change yet, just new unused models) — full suite, not just a smoke import.
- Commit: feat: add Lead and CommunicationLog models

Task 1.2: CRM Migration (0003)

Files: Create backend/migrations/versions/0003_crm_schema.py.

- down_revision = "0002".

- CREATE TABLE leads and communication_logs exactly per [Database Schema](#) Section 3, including the status CHECK constraint ('new', 'contacted', 'estimating', 'qualified', 'won', 'lost') and channel CHECK constraint ('call', 'email', 'note', 'sms').
- CREATE INDEX idx_leads_company_status ON leads(company_id, status).
- Enable RLS on both tables. Add tenant_isolation policy on each, FOR ALL, USING/WITH CHECK both using the guarded pattern (inherited invariant #1): company_id IN (SELECT id FROM get_all_descendant_ids(NULLIF(current_setting('app.current_tenant', true), '')::uuid)).
- REVOKE UPDATE, DELETE ON communication_logs FROM app_user (design decision #6). Confirm app_user still has SELECT, INSERT (it should, from Task 5's blanket grant — verify, don't assume).
- Verify by hand against live Postgres, the same way Phase 0's Task 5 did: apply the migration, confirm \d+ leads/\d+ communication_logs show RLS enabled and the expected policies, confirm app_user genuinely cannot UPDATE/DELETE a communication_logs row (expect a permissions error, not a silent no-op).
- Run the full suite (should still be green — nothing references these tables yet).
- Commit: feat: add CRM schema migration (leads, communication_logs)

Task 1.3: Lead & CommunicationLog Schemas

Files: Create backend/app/schemas/lead.py, backend/app/schemas/communication_log.py.

- LeadCreateRequest: contact_name, project_name, email (EmailStr), phone (optional), project_type, estimated_value (optional), notes (optional). Apply Field(..., min_length=...) constraints matching the rigor Phase 0 applied to RegisterRequest/InvitationAcceptRequest — don't ship an unconstrained schema this time and have code review catch it later.
- LeadUpdateRequest: all fields optional (PATCH semantics) except status, which is handled by its own state-machine validation in the router, not blind field assignment (see Task 1.5).
- LeadResponse: full model, ConfigDict(from_attributes=True).
- CommunicationLogCreateRequest: channel, body.
- CommunicationLogResponse: full model + author_id.
- Verify: full suite green.
- Commit: feat: add Lead and CommunicationLog Pydantic schemas

Task 1.4: POST /leads, GET /leads, GET /leads/{id}

Files: Create backend/app/routers/leads.py. Modify backend/app/main.py. Create backend/tests/test_leads.py.

- POST /leads: require_role("admin", "project_manager") per the RBAC matrix (CRM = Admin + PM full CRUD, nobody else). Creates a Lead with status="new", company_id=current.company_id. Reuses current.session, no explicit commit (inherited invariant #4). Writes an audit_log entry (lead.created), following the exact pattern from Phase 0's create_child_company/create_invitation.
- GET /leads: same roles. Pagination per [API Specification](#) Section 1 (?limit=, default 25, max 100, ?cursor=) — this is the first paginated list endpoint in the project; implement it once, cleanly, since Projects/Tasks/Documents will all reuse the same pattern. ?status=filter.
- GET /leads/{id}: same roles, 404 for cross-tenant (RLS-backed, same pattern as GET /companies/{id}).
- Tests: create, list, list-with-status-filter, list-pagination (cursor correctness — seed >1 page worth), get-own, get-cross-tenant-404 (mirrors test_tenant_isolation.py's pattern exactly), get-nonexistent-404, non-admin/PM roles blocked (403) — reuse the "insert a real non-admin membership directly via SQL" pattern from Phase 0's test_deps.py if a real field_crew/client user is needed and no invitation-based path exists yet to create one through the API.
- Verify: full suite green.
- Commit: feat: add POST/GET /leads with pagination and RBAC

Task 1.5: Lead Status State Machine (PATCH /leads/{id})

Files: Create backend/app/services/lead_transitions.py. Modify backend/app/routers/leads.py. Create backend/tests/test_lead_state_machine.py.

- Legal transitions, table-driven (per [Test Strategy](#) Section 4): new→contacted, contacted→estimating, estimating→qualified, qualified→won, qualified→lost, and any status →lost (a lead can be lost from most stages, not just qualified — confirm this against [Functional Requirements](#) US-2.2's stated pipeline order and use judgment on which "lost" transitions are legal; document the chosen transition table explicitly in a comment, since the functional-requirements doc states the pipeline linearly but doesn't fully enumerate "lost" as a fork from every stage). Illegal transitions (e.g. new→won skipping stages, or any transition out of a terminal won/lost state) return 409, matching the API spec's error-code conventions.

- Status transitions are logged: `updated_at` bumped, and an `audit_log` entry (`lead.status_changed`, `metadata {from, to}`) per [Security & Compliance](#) Section 5 (“every state-changing action... writes an `audit_log` row”).
- **Business rule from [Functional Requirements](#) Section 2:** “A Lead cannot be deleted once it has an associated Estimate or Project; it can only be marked Lost.” There’s no `DELETE /leads/{id}` route in the API spec at all, so this rule is enforced by omission (no delete route exists) rather than needing an explicit check — note this in a comment so a future task adding a delete route doesn’t skip re-deriving this rule.
- **won is the trigger for Task 1.7’s LEAD_WON event** — this task publishes the event but the Project Management consumer isn’t wired until Task 1.7; keep the `publish()` call itself in this task (it’s a no-op with zero registered handlers until then) so Task 1.7 is purely “add a handler,” not “add the publish call too.”
- Non-status fields (`contact_name`, `phone`, `notes`, etc.) are also PATCH-able via this same route (per API spec: “Update Lead fields / status,” combined) — validate both together; don’t let a request patch both a valid field and an illegal status transition and have one silently fail while the other succeeds (should be one transaction, one outcome).
- Tests: every legal transition succeeds, every illegal transition returns 409 with no state change, plain-field-only PATCH (no status change) succeeds, combined field+status PATCH, audit log entry written and content-correct for a status change (following the assertion style of Phase 0’s `test_audit_log.py`).
- Verify: full suite green.
- Commit: feat: add Lead status state machine with audit logging

Task 1.6: In-Process Event Bus

Files: Create `backend/app/core/events.py`. Create `backend/tests/test_events.py`.

- Design decision #2’s dispatcher. Minimal surface: `register(event_name: str, handler: Callable)` and `publish(event_name: str, **payload)`, calling every registered handler synchronously, in registration order, propagating the first handler’s exception (don’t swallow errors — an event handler failing mid-request should roll back the whole transaction, matching the ACID expectations Phase 0 established for `get_current_user`).
- No dependency on Redis/Celery. A plain in-process module-level registry is sufficient and testable in isolation.
- Tests: `register` + `publish` calls the handler with the right payload; multiple handlers for the same event all fire; an unhandled event name is a no-op, not an error (nothing is required to subscribe); a handler’s exception propagates to the caller.

- Verify: full suite green.
- Commit: feat: add in-process event bus (design decision #2)

Task 1.7: POST /leads/{id}/communications, GET /leads/{id}/communications

Files: Modify backend/app/routers/leads.py. Create backend/tests/test_communication_logs.py.

- POST: require_role("admin", "project_manager"), author_id=current.user.id, company_id=current.company_id. No update/delete route — immutability is both DB-enforced (Task 1.2's REVOKE) and API-shape-enforced (route doesn't exist).
- GET: paginated, chronological (oldest or newest first — check [Functional Requirements](#) US-2.4: “see a chronological history,” pick oldest-first as the natural reading order for a history log unless there's a stronger signal otherwise).
- Tests: create + list, list ordering, cross-tenant 404 on lead_id, attempt a raw DB UPDATE/DELETE as app_user directly (bypassing the API entirely) and confirm the REVOKE from Task 1.2 actually blocks it — this is the same “prove the mechanism, not just the app-layer behavior” discipline as Phase 0's RLS regression tests.
- Verify: full suite green.
- Commit: feat: add communication log create/list with DB-enforced immutability

Task 1.8: CRM Tenant-Isolation Regression Tests

Files: Create backend/tests/test_tenant_isolation_phase1.py (or extend the existing test_tenant_isolation.py — decide based on which reads more cleanly; note the choice in the commit message).

- Mirror Phase 0's Task 12/16 rigor for leads and communication_logs: direct-ID cross-tenant access blocked (404), header-spoofing blocked (403), and at least one RLS-disable/re-enable regression test (Phase 0's Task 16 pattern) proving the *policy*, not app-layer filtering, is what blocks access — for leads specifically, since it's the new table with the most exposure (list + get + patch + create-comm-log-under-it).
 - Verify: full suite green, run twice for stability (matches Task 16's own verification discipline).
 - Commit: test: add tenant-isolation regression coverage for CRM tables
-

Task 1.9: Project, Phase, Task, Document, DailyLog Models

Files: Create backend/app/models/project.py, phase.py, task.py, document.py, daily_log.py. Modify backend/app/models/__init__.py.

- Match [Database Schema](#) Section 4 exactly for every column/type/nullability/FK, **except**: do not create a change_orders model (out of scope, see top of this doc).
- Project.lead_id is nullable (a Project can be manually initialized without a Lead, per POST /projects's documented inputs).
- Verify: full suite green.
- Commit: feat: add Project, Phase, Task, Document, DailyLog models

Task 1.10: Project Management Migration (0004)

Files: Create backend/migrations/versions/0004_project_management_schema.py.

- down_revision = "0003".
- CREATE TABLE projects, phases, tasks, documents, daily_logs per the schema doc, **excluding change_orders**. Every table gets tenant_isolation FOR ALL with the guarded USING/WITH CHECK pattern (inherited invariant #1/#2).
- REVOKE UPDATE, DELETE ON daily_logs, documents FROM app_user (design decision #6). documents needs INSERT/SELECT only — a new version is a new row.
- tasks and projects DO need UPDATE (status changes, assignment changes) — don't revoke there.
- Verify by hand against live Postgres per the same discipline as Task 1.2 (confirm RLS + policies + grants actually landed, don't just trust the migration ran without error).
- Run the full suite (still green — nothing wired up yet).
- Commit: feat: add Project Management schema migration

Task 1.11: Project/Phase/Task/Document/DailyLog Schemas

Files: Create backend/app/schemas/project.py, phase.py, task.py, document.py, daily_log.py.

- ProjectCreateRequest: name, site_address, lead_id (optional), projected_start_date (optional).
- ProjectResponse: full model.
- ProjectClientDashboardResponse: id, name, status, site_address, projected_start_date, phase_count, task_count, completed_task_count

(design decision #8's correction) — no `lead_id` (internal CRM linkage, not the client's business), matching design decision #8's "no internal operational detail" framing.

- `ProjectStatusUpdateRequest`: status, optional reason.
- `ProjectPatchRequest` (design decision #3): name, site_address, projected_start_date, all optional.
- `PhaseCreateRequest/PhaseResponse`: name, sequence.
- `TaskCreateRequest`: name, due_date (optional), assignee_id (optional), phase_id. `TaskUpdateRequest`: status, assignee_id, both optional. `TaskResponse`.
- `DocumentResponse` only — no JSON upload-request schema: the upload itself is multipart/form-data, not a JSON body (FastAPI UploadFile + a file_name form field, handled directly in Task 1.15's router).
- `DailyLogCreateRequest`: log_date, weather (optional), notes (optional). Note: [Functional Requirements](#) US-3.3 mentions photos; documents is the only file-storage mechanism built in Phase 1 — decide whether Daily Log photos are out of scope for Phase 1 (simplest: yes, defer photo attachment to a later pass, ship text-only Daily Logs first) or reuse the documents upload path with a daily_log_id linkage the schema doc doesn't currently have. **Recommendation: defer photo attachment**, ship Daily Logs without it — the schema doc's daily_logs table has no photo/document linkage column at all, so adding one now is schema-doc scope creep for a P1-nice-to-have, not the stated exit criterion.
- Verify: full suite green.
- Commit: feat: add Project Management Pydantic schemas

Task 1.12: POST/GET /projects, GET /projects/{id}, PATCH /projects/{id}

Files: Create backend/app/routers/projects.py. Modify backend/app/main.py. Create backend/tests/test_projects.py.

- POST /projects: require_role("admin", "project_manager"). Manual initialization path (independent of LEAD_WON — Task 1.18 wires the automatic path).
- GET /projects: paginated, ?status= filter, same roles plus read access per the RBAC matrix (field_crew reads *assigned* projects only — this needs a join through tasks.assignee_id, more restrictive than a blanket company-scoped list; accountant reads all, financial-fields-only, but Phase 1 has no financial fields on projects yet so this collapses to plain read access for now; client gets the sanitized dashboard shape, not the list route at all, per the API spec only documenting GET /projects/{id} for clients).

- GET /projects/{id}: role-based response shape per design decision #8. 404 for cross-tenant. For the client-role shape, compute phase_count/task_count/completed_task_count via COUNT queries against phases/tasks scoped to this project_id (both tables already exist as of Task 1.10's migration — no dependency on Task 1.14's own routes existing yet).
- PATCH /projects/{id} (design decision #3): require_role("admin", "project_manager").
- Tests: create, list, list-filtered-by-status, list-as-field-crew-only-shows-assigned (this is the one genuinely new RBAC-shape test in this task — write it carefully), get-full-vs-get-as-client-sanitized (assert the client response genuinely omits lead_id and any field not in ProjectClientDashboardResponse, not just that it returns 200), patch, cross-tenant 404s across all of the above.
- Verify: full suite green.
- Commit: feat: add Project CRUD, RBAC-scoped listing, and client dashboard view

Task 1.13: Project Status State Machine (PATCH /projects/{id}/status)

Files: Create backend/app/services/project_transitions.py. Modify backend/app/routers/projects.py. Create backend/tests/test_project_state_machine.py.

- Legal transitions per [Functional Requirements](#) US-3.2: draft→pre_construction→active→suspended→completed→archived, plus active→suspended and suspended→active (suspension needs to be reversible — the linear list alone doesn't capture that, use judgment and document the actual transition table in a comment, same as Task 1.5's Lead state machine). Illegal transitions (e.g. draft→completed) → 409.
- **Business rule:** "A Project cannot move to Completed while it has open (non-approved) Change Orders" ([Functional Requirements](#) Section 3) — **not enforceable in Phase 1** since change_orders doesn't exist yet (out of scope, see top of doc). Add a code comment at the completed transition noting this check needs to be added when Change Orders ships in Phase 2, so it isn't silently forgotten.
- Audit log entry (project.status_changed, metadata {from, to, reason}) per Security & Compliance Section 5.
- Tests: table-driven legal/illegal transitions (per Test Strategy Section 4), suspend/resume round-trip, audit log entry correctness.
- Verify: full suite green.
- Commit: feat: add Project status state machine with audit logging

Task 1.14: Phases and Tasks

Files: Modify backend/app/routers/projects.py (or split into backend/app/routers/tasks.py if the file is getting unwieldy — use judgment). Create backend/tests/test_phases_tasks.py.

- POST /projects/{id}/phases: require_role("admin", "project_manager").
- POST /projects/{id}/tasks: same roles. phase_id must belong to the same project (application-layer check, same pattern as Phase 0's create_child_company's company_id != current.company_id guard — verify cross-project phase assignment is actually rejected, not just cross-tenant).
- PATCH /tasks/{id}: admin/project_manager can update anything; field_crew can update status only on tasks assigned to them (assignee_id == current.user.id) — this is a genuinely new RBAC shape (role AND ownership, not just role), test it explicitly both ways (field_crew updating their own task succeeds, field_crew updating someone else's task is blocked). **Correction, found during this task's spec review:** "blocked" for a not-assigned-to-them task is **404**, not 403 as this bullet originally said — docs/07-security-compliance.md Section 2 gives field_crew unqualified "Read assigned" for Project Management, so a task they're not assigned to isn't a restricted action on a visible thing, it's invisible entirely, matching the established _get_project_or_404/_get_lead_or_404 "404 = doesn't exist or isn't yours, indistinguishable" convention used everywhere else in this codebase. A field_crew user attempting to PATCH a field they ARE allowed to see but not touch (e.g. assignee_id on their own assigned task) gets 403 — that's the case this bullet's "blocked" language actually describes correctly. In short: 404 for "can't see it," 403 for "can see it but can't touch that field."
- Tests: phase creation, task creation, cross-project phase_id rejected (422 — a semantic FK-context mismatch on a fully-visible, same-tenant phase, not an authorization failure, so distinct from create_child_company's 403 for genuinely cross-tenant access), task patch as PM (any task), task patch as field_crew (own task, status-only succeeds; own task with assignee_id in the payload is 403 even if set to its current value, not silently dropped; a task not assigned to them is 404, byte-identical to a genuinely nonexistent task_id), cross-tenant 404s.
- Verify: full suite green.
- Commit: feat: add Phases and Tasks with assignee-scoped field-crew permissions

Task 1.15: Documents (local filesystem storage)

Files: Create backend/app/services/document_storage.py. Modify backend/app/routers/projects.py, backend/app/config.py (add storage_root setting), docker-compose.yml (documents volume, design decision #4). Create backend/tests/test_documents.py.

- `POST /projects/{id}/documents`: `require_role("admin", "project_manager")`. Accepts multipart/form-data. Writes the file under `{STORAGE_ROOT}/{company_id}/{project_id}/{file_name}`; if a document with the same `file_name` already exists for this project, insert a new row with `version = previous_max + 1`, don't overwrite the file on disk (both versions must remain retrievable per US-3.4's "prior versions accessible").
- Validate `file_name` doesn't allow path traversal (`../`, absolute paths) before it ever touches the filesystem — this is the first user-controlled filesystem path in the project, treat it with the same suspicion Phase 0 applied to the first user-controlled UUID (X-Tenant-ID) in design decision #10's spirit; write a test specifically for a `file_name` like `../../../../etc/passwd` and confirm it's rejected, not sanitized-and-allowed.
- **Correction, found during this task's spec review:** the literal `{STORAGE_ROOT}/{company_id}/{project_id}/{file_name}` path template above is unsatisfiable together with this same bullet's "don't overwrite the file on disk (both versions must remain retrievable)" requirement — two uploads sharing a `file_name` would collide on the identical path. The implementation resolves this by nesting each version under its own path segment, `{company_id}/{project_id}/{version}/{file_name}` (applied uniformly, including to version 1, so there's no special-casing of "first upload"), keeping `storage_path` collision-free by construction and the original `file_name` preserved byte-for-byte. Also found during spec review: `validate_file_name` initially rejected `..`, path separators, and absolute paths but not embedded control characters — a `file_name` containing a NUL byte reached the database layer unrejected and crashed with an unhandled 500 (Postgres' UTF8 text type itself rejects NUL) instead of the intended clean 422. Fixed by rejecting any character below `0x20` outright, same "reject, don't sanitize" treatment as every other check in `validate_file_name`.
- `GET /projects/{id}/documents` (list, most recent version per filename shown by default per US-3.4) — not explicitly in the API spec table but necessary to satisfy "the most recent version is shown by default with prior versions accessible"; add it, matching the same "spec is conceptual, not exhaustive" reasoning as design decision #3.
- **Correction, found during this task's code-quality review:** two further gaps in the same family as the spec-review corrections above, both fixed. (1) `validate_file_name` had no length check against

documents.file_name's String(255) column width, so an over-long name reached the DB insert unrejected and crashed with an unhandled 500 instead of a 422 — fixed by rejecting any file_name over 255 characters. (2) write_document_file's original existence check (.exists()) then .write_bytes()) had a check-then-write gap: two genuinely concurrent uploads of the same file_name can legitimately compute the same next version (the previous_max_version read has no locking), and the loser could silently overwrite the winner's content instead of failing. Fixed by writing with exclusive-create ("xb" mode, atomic at the OS level) and mapping the resulting FileExistsError to a 409 Conflict in upload_document — the caller can retry, and no content is ever silently lost.

- Tests: upload, upload-new-version-of-same-filename (both retrievable), path-traversal-rejected (including a control-character/NUL-byte variant and an over-length variant), concurrent-upload-of-same-filename (409 or two distinct versions, never a silent overwrite or a 500), cross-tenant 404, nonexistent-project 404, empty-file-content upload, list shows latest-version-only by default.
- Verify: full suite green.
- Commit: feat: add Document upload with versioning and local filesystem storage

Task 1.16: Daily Logs

Files: Modify backend/app/routers/projects.py. Create backend/tests/test_daily_logs.py.

- POST /projects/{id}/daily-logs: require_role("admin", "project_manager", "field_crew") per the RBAC matrix (Field Crew "create Daily Logs" explicitly). author_id = current.user.id. No update/delete route (immutability, DB-enforced per Task 1.10's REVOKE).
- GET /projects/{id}/daily-logs: paginated, same read roles as project detail.
- Tests: create as PM, create as field_crew, create as client blocked (403), immutability (raw UPDATE/DELETE as app_user blocked, same discipline as Task 1.7), cross-tenant 404.
- Verify: full suite green.
- Commit: feat: add Daily Logs with field-crew write access and DB-enforced immutability

Task 1.17: Project Management Tenant-Isolation Regression Tests

Files: Extend backend/tests/test_tenant_isolation_phase1.py.

- Same rigor as Task 1.8, applied to projects, phases, tasks, documents, daily_logs. Include the parent/child-branch hierarchy case explicitly (Phase 0's Task 13 pattern: parent sees child-branch projects, siblings don't see each other) since Project Management is exactly the kind of module where hierarchical visibility (per US-3.1's "hierarchical visibility" business rule) actually matters in practice, unlike CRM which didn't call this out explicitly.
 - Verify: full suite green, run twice.
 - Commit: test: add tenant-isolation regression coverage for Project Management tables
-

Task 1.18: LEAD_WON → Draft Project Wiring

Files: Modify backend/app/services/lead_transitions.py (register the handler), create backend/app/services/lead_won_handler.py (or inline in project_transitions.py — use judgment on which reads more cleanly given the event bus's register() call needs to live somewhere at app startup, likely app/main.py or a dedicated app/core/event_handlers.py that both modules' routers import). Create backend/tests/test_lead_won_drafts_project.py.

- On Lead.status transitioning to won (Task 1.5), publish LEAD_WON with {lead_id, company_id, contact_name, project_name}.
- Handler creates a Project with name=lead.project_name, lead_id=lead.id, company_id=lead.company_id, site_address='' (design decision #5), status='draft'. Writes an audit_log entry (project.drafted_from_lead).
- **This must happen in the same transaction as the Lead status update** — inherited invariant #4 (reuse current.session, single commit owned by the dependency) applies here too; the event handler receives the same session the route handler is using, not a new one. If the Lead update succeeds but the Project draft fails, both must roll back together — write a test that forces the handler to raise and confirms the Lead's status change didn't persist either (mirrors the rigor of Phase 0's Task 11 rollback regression test).
- Tests: won transition creates exactly one Project with correct carried-over fields; the Project's site_address is ''; a failure in the handler rolls back the Lead status change too; other Lead status transitions (contacted, qualified, etc.) do NOT trigger project creation — only won does.
- Verify: full suite green.
- Commit: feat: wire LEAD_WON event to auto-draft a Project

Task 1.19: Full-Stack E2E Extension

Files: Modify `scripts/e2e_smoke_test.py`.

- Extend the existing live-stack smoke test (Phase 0's Task 19 script) with the Phase 1 exit criterion itself, over real HTTP: register a company, create a Lead, walk it through the full legal transition chain to won, assert a Project now exists referencing that Lead with the carried-over `project_name`, assert its `site_address` is '' as expected, assert a client-role user can GET the drafted project and receives the sanitized shape (no `lead_id` in the response). This is literally the Phase 1 roadmap exit criterion, run against real containers, not just in-process tests.
- Run against the full local stack per Phase 0's Task 19 procedure (docker compose up -d --build, alembic upgrade head, run the script, docker compose down without -v).
- Commit: test: extend E2E smoke test with Lead-to-Project flow (Phase 1 exit criterion)

Task 1.20: Full Regression Pass + Plan Closeout

Files: None (verification only) — update `docs/superpowers/plans/2026-07-08-phase-1-crm-project-management.md`'s exit-criteria checklist below.

- Full `pytest -v` from `backend/`, run twice for stability.
 - Full RLS regression suite (`test_tenant_isolation_phase1.py` + Phase 0's original `test_tenant_isolation.py` + `test_rls_policy_regression.py`) — confirm no regression in Phase 0's own tables from anything Phase 1 touched (it shouldn't have touched them, but verify rather than assume).
 - CI green on a real GitHub Actions run (same discipline as Phase 0's Task 17 — open a verification PR if needed, confirm the run, then close/clean it up).
 - Commit: docs: close out Phase 1 exit criteria checklist
-

Phase 1 Exit Criteria Checklist

- ☒ Lead CRUD, pipeline status transitions (state-machine enforced, illegal transitions rejected), communication logs (immutable, DB-enforced) — Tasks 1.1-1.8
- ☒ Project CRUD/lifecycle state machine, Phases, Tasks (assignee-scoped RBAC), Documents (versioned, local storage), Daily Logs (immutable, DB-enforced) — Tasks 1.9-1.17
- ☒ LEAD_WON → draft Project event wiring, transactionally consistent — Task 1.18

- ☒ Client-facing read-only project dashboard (sanitized view, RBAC-scoped) — Task 1.12
- ☒ A Lead can be created, moved to Won, and lands as a Draft Project with client details carried over, end-to-end, with tests, verified over real HTTP against real containers — Task 1.19
- ☒ Tenant isolation proven on every new table, including the parent/child hierarchy case — Tasks 1.8, 1.17
- ☒ Full regression suite green in CI — Task 1.20 (247 backend tests passing, run twice for stability; full RLS regression suite — test_tenant_isolation.py + test_rls_policy_regression.py + test_tenant_isolation_phase1.py — 24/24 passing; CI verification PR #3 green, closed without merging pending the user's explicit merge decision)